

Customer Churn Platform

Complete Project Guide

Welcome to the Complete Project Guide for the Customer Churn Prediction Platform. This guide is designed to teach you everything about this project from absolute scratch.

We will cover the business problem, the technologies, the libraries, the machine learning algorithms, the API, the dashboard, and a detailed code walkthrough for every single file.

Part 1: What is Customer Churn? (The Business Problem)

What is Churn?

Customer churn (or customer attrition) refers to when a customer stops doing business with a company. For a subscription-based business (like a telecom carrier, streaming service, or SaaS application), churn is one of the most critical business metrics.

Why Predict Churn?

Acquiring new customers is expensive (marketing, onboarding, sales). Retaining existing customers is far more cost-effective. By using historical data, we can train a machine learning model to predict the probability that a customer will cancel their subscription *before* they do it.

This allows the marketing or customer success teams to intervene proactively. For example, if the model predicts a customer has an 85% chance of churning, the company can send them a discount code, offer a free upgrade, or reach out to resolve technical issues.

Part 2: Core Libraries & Why We Use Them

Here is a list of the core libraries used in this project and why they are necessary:

1. **Pandas:** A data manipulation library. It provides "DataFrames" (2D tables, like Excel sheets in code). We use it to load, clean, join, and analyze customer files.
2. **NumPy:** A library for numerical computations. It supports large arrays and mathematical functions. We use it to represent missing values (`np.nan`) and perform math operations.
3. **Scikit-Learn:** The industry-standard library for traditional machine learning in Python. We use it for:
 - o Data preprocessing: `StandardScaler` (scales numbers) and `OneHotEncoder` (converts text categories to numbers).
 - o Splitting data: `train_test_split` to divide data into training and test sets.

- o Algorithms: `LogisticRegression` and `RandomForestClassifier`.
 - o Evaluation: Metrics like accuracy, precision, recall, and ROC-AUC.
4. XGBoost: A specialized library that implements Gradient Boosted Decision Trees. It is famous for winning machine learning competitions and is incredibly fast and accurate.
 5. SQLAlchemy & Sqlite3: Used for database connectivity. Sqlite3 is a lightweight SQL database built into Python. SQLAlchemy provides a uniform interface to run SQL queries in Python.
 6. FastAPI: A modern, high-performance web framework for building APIs. We use it to serve our machine learning predictions over the web.
 7. Uvicorn: An ASGI web server that runs FastAPI applications.
 8. Streamlit: A framework that lets us build web dashboards in pure Python. It is perfect for data science teams to show their models without needing HTML/CSS/JavaScript.
 9. Joblib: Used to save and load serialized Python objects (like our trained models and preprocessing pipelines).
 10. Matplotlib & Seaborn: Data visualization libraries. We use them to generate correlation countplots, density charts, ROC curves, and confusion matrices.
 11. Pytest & Requests: Pytest is a framework to write automated tests. Requests is an HTTP library used to call endpoints.

Part 3: Understanding Machine Learning Algorithms

In this project, we train and compare three distinct classification models. Here is why we use them and how they work:

1. Logistic Regression

- o What it is: Despite the name "regression," it is a classification algorithm. It calculates the relationship between features and the probability of a binary outcome (Churn = 1 or 0) using a logistic (sigmoid) function.
- o Why we use it: It is simple, extremely fast to train, and highly interpretable. It serves as our "baseline." If a complex model cannot beat Logistic Regression, we stick to the simpler model.

2. Random Forest Classifier

- o What it is: An ensemble algorithm. It builds multiple Decision Trees (each trained on a random subset of data and features) and combines their predictions (voting) to get a final result.
- o Why we use it: It is highly robust, handles non-linear relationships naturally, does not require numeric scaling to perform well, and provides "Feature Importances" to show which factors drive decisions.

3. XGBoost Classifier

- o What it is: Short for "Extreme Gradient Boosting." It builds decision trees sequentially, where each new tree is trained to correct the errors made by the previous trees.
- o Why we use it: It provides state-of-the-art accuracy on tabular data, includes built-in regularization to prevent overfitting, and runs extremely fast due to parallel computing optimizations.

Part 4: What is an API and FastAPI?

What is an API?

An API (Application Programming Interface) allows different software programs to talk to each other.

In our project, the machine learning model is like a brain. The API wraps this brain with Web Endpoints. When a customer enters details on our dashboard, the dashboard sends an HTTP request (a message) to the API. The API feeds the details to the model, gets the prediction, and sends it back to the dashboard.

Why FastAPI?

1. Speed: It is as fast as Node.js and Go.
2. Automatic Docs: It generates interactive documentation (Swagger UI) automatically at `/docs``.
3. Data Validation: It uses Pydantic to validate that incoming data has correct types (e.g., tenure must be an integer, monthly charges must be a float).

Part 5: Line-by-Line Code Walkthrough

Below is an explanation of every single file in the project, explaining what the code does and why it is written that way.

1. Data Ingestion (`src/data_ingestion.py``)

This file is responsible for merging the CSV demographics and the SQL database services.

- o Lines 1-7: Standard imports. We import `pandas`` to handle tabular data, `sqlite3`` to connect to our DB, and `logging`` to print status messages to the console.
- o `ingest_data(csv_path, db_path)``:
- o Checks if the CSV and DB files exist.
- o `pd.read_csv(csv_path)``: Loads the demographics CSV data.
- o `sqlite3.connect(db_path)``: Connects to the SQLite database.
- o `pd.read_sql_query("SELECT * FROM services", conn)``: Runs a SQL query to load all service details into a DataFrame.
- o `pd.merge(df_demographics, df_services, on="CustomerId", how="inner")``: Joins the two tables on the common key `CustomerId``. An inner join ensures only matching records are returned.

2. Preprocessing & Feature Engineering (`src/preprocessing.py``)

This file cleans the data and sets up the preprocessing pipeline.

- o Numerical vs. Categorical Lists: We define lists of numeric columns (`Tenure``, `MonthlyCharges``, `TotalCharges``) and categorical columns (`Gender``, `Contract``, etc.).
- o `clean_data(df)``:
- o `df["TotalCharges"].replace(r'^\s*$', np.nan, regex=True)``: Replaces whitespace in `TotalCharges`` (representing new customers who haven't paid yet) with NaN.
- o `pd.to_numeric(...)``: Converts `TotalCharges`` to float type.
- o `df["TotalCharges"].fillna(...)``: Imputes missing charges by multiplying `MonthlyCharges`` by `Tenure``.
- o `df["Churn"].map({"Yes": 1, "No": 0})``: Maps target strings to binary integers.
- o `build_preprocessing_pipeline()``:

- o Creates a `ColumnTransformer`. For numerical columns, we apply `SimpleImputer` (fills missing with median) and `StandardScaler` (normalizes values to mean=0, std=1).
- o For categorical columns, we apply `OneHotEncoder` (converts text categories into separate columns of 0/1, ignoring unseen values in the future).
- o `fit_and_save_pipeline(...)`: Fits this pipeline on the raw data and saves it to `models/preprocessor.joblib` using `joblib`.

3. Model Training (`src/train.py`)

Trains and compares the models.

- o `train_test_split(...)`: Divides the preprocessed features (X) and labels (y) into 80% training and 20% test. `stratify=y` guarantees that the training and test sets have the same ratio of churned vs. non-churned customers.
- o Model Dictionary: Instantiates `LogisticRegression`, `RandomForestClassifier`, and `XGBClassifier`.
- o Training Loop:
- o `model.fit(X_train, y_train)`: Trains the algorithm.
- o `model.predict(X_test)` and `predict_proba(X_test)`: Generates predictions and probabilities.
- o Evaluates scores (Accuracy, Precision, Recall, ROC-AUC).
- o Selects the model with the highest ROC-AUC score and saves it as `models/best_model.joblib`.
- o Exports metrics to `models/metrics.json`.

4. API Backend (`src/api.py`)

The web service that hosts our model.

- o Lifespan Manager (`lifespan`):
- o The `@asynccontextmanager` loads the model (`best_model.joblib`) and preprocessor (`preprocessor.joblib`) once when the API server starts up. This saves memory and makes incoming requests extremely fast.
- o Pydantic Model (`CustomerPayload`):
- o Defines the exact fields and data types expected from clients. If a client sends a string for `MonthlyCharges` instead of a float, FastAPI will catch it and return an error before running predictions.
- o `/health` Endpoint: Returns the name of the best model and server status.
- o `/predict` Endpoint:
- o Accepts a list of customer data payloads.
- o Converts payload items to a DataFrame using `item.model_dump()`.
- o Runs `clean_data` and transforms features using the preprocessor.
- o Runs the model (`predict_proba` and `predict`) to calculate probability and class.
- o Returns a JSON response containing the probabilities.

5. Streamlit Dashboard (`src/dashboard.py`)

The user interface.

- o `sys.path.append(...)`: Adds the workspace folder to Python's search path so Streamlit can locate our preprocessing files.
- o `load_raw_stats()`: Connects to the SQLite DB to calculate statistics (KPI cards) and load charts.
- o Predictor fallbacks:
- o `predict_churn_api()`: Sends a POST request to the FastAPI server at `http://localhost:8000/predict`.

- o `predict_churn_local()`: If the FastAPI server is offline, this function automatically loads the models locally and calculates the prediction. This makes the dashboard extremely resilient!
- o Sidebar: Provides page routing (`Overview & Insights`, `Single Customer Predictor`, and `Model Diagnostic Hub`).
- o KPI & Visuals: Renders charts (countplot and kdeplot) using Matplotlib and custom CSS for cards.
- o Customer Predictor form: Sets up a form with input fields for all features and displays a color-coded gauge bar for the churn risk.

6. Containerization (Dockerfile & docker-compose.yml)

- o Dockerfile: Packages the app. It starts with Python 3.11, installs OS compiler tools, runs `pip install -r requirements.txt`, copies our files, and exposes ports 8000 and 8501.
- o docker-compose.yml: Connects both containers.
- o `api`: Starts FastAPI on port 8000.
- o `dashboard`: Starts Streamlit on port 8505.
- o Volumes (`./data` and `./models`): Shares local folders with the containers so training metrics are synchronised.